

차량과 코드를 잇는 4년.

안드로이드 엔지니어 / 인포카 · OBD2 · 블루투스 · 안드로이드 오토

● 누적 다운로드

1,000만+

인포카 (B2C) 안드로이드 서비스

● 경력

3년 10개월

인포카 안드로이드 · 2022.08 ~ 재직중

● 주요 프로젝트

5개

AI · OBD · 광고 · 오토 · 차계부

목차

/ INDEX

자기소개부터 5개 주요 프로젝트까지 — 차량 도메인에서 4년간 쌓은 기록.

총 페이지

23_p

소개 섹션

4_{파트}

주요 프로젝트

5_개

도메인

차량 · OBD

소개 · INTRO

03	자기소개	개발자 소개	03
04	경력 사항	한 회사 · 한 도메인	04
05	기술 보유	언어 · 도구 · 아키텍처	05
06	참여 프로젝트 목록		06
마무리 · OUTRO			
23	감사 인사		23

주요 프로젝트 · WORK

01	AI 워크플로우 기반 개발	knot · MCP · 점진 전환	07-10
02	프리셋 대시보드 (OBD)	5종 엔진 × 3종 대시보드	11-14
03	광고 미디어이션 전환	5개 네트워크 워터폴	15-17
04	안드로이드 오토 확장	Car App Library	18-20
05	차계부 DB 리뉴얼	정규화 · 마이그레이션	21-22

자기소개.



빠르게 변하는 기술을 **실제 업무에 녹여내는 것**을 중요하게 생각하는 개발자입니다.

1,000만 다운로드 서비스의 사용자 접점부터 **운영·유지보수**까지 다루며, 최근엔 **AI로 저의 개발 방식**을 새롭게 다듬고 있습니다.

01

사용자 접점 기능

1,000만 다운로드 서비스의 블루투스 통신·푸시·알림 센터 등 일상 접점 기능 개발·운영.

02

레거시 현대화

오래된 MVC(Java) 코드를 Kotlin·Hilt·MVVM 구조로 점진적으로 전환.

03

AI 워크플로우

요구사항→명세→구현→검증 과정을 AI 도구와 함께 진행하는 작업 방식을 정착.

04

운영·유지보수

릴리즈 모니터링·NPE 트러블슈팅·바인딩 리팩토링
·Play Console 정책 대응
·CS 분석까지.

05

투명한 협업

PM·디자이너·iOS와 스크럼, 기술 제약을 투명하게 공유·조율. iOS 크로스체크로 정책 통일.

소속

(주) 인포카
infoCar Inc.

직책

전임 연구원
주임 → 전임

기간

3년 10개월
2022.08 — 재직중

서비스

B2C · B2B
인포카 · 인포카 비즈

도메인

OBD2 · 블루투스
안드로이드 오토 · 광고

학력

신한대학교
컴퓨터공학과 졸업

경력 사항.

2026.01 - 2026.05 · 진행 중

AI 워크플로우 기반 개발 · 레거시 현대화

요구사항→명세→구현→검증을 AI 도구와 함께 · MVC(Java) → Kotlin 점진 전환

2025.09 - 2025.12

프리셋 대시보드 (OBD 차량 데이터)

B2B(인포카 비즈) 기능을 B2C에 이식 · 5종 엔진 × 3종 대시보드 · 설정 자동 복구 + 데이터 마이그레이션

2025.03 - 2025.05

광고 미디어이션 전환

단일 AdMob → 5개 네트워크 워터폴 · Firebase Remote Config 9개 항목 동적 제어 · 광고 매출 ▲475%

2024.10 - 2025.02

안드로이드 오토 플랫폼 확장

Car App Library 신규 개발 · 앱-오토 화면 동기화 · 앱 종료 후에도 단독 동작 · 상태별 안내 화면

2024.06 - 2024.08

본앱 사용성 리뉴얼

차량 등록 방식 리뉴얼 · "운전→주행" 다국어 UX 통일 · 주유소·세차장·주차장 검색 · 영수증 API 리팩토링

2024.01 - 2024.05

제조사 데이터·Pro 구독 정책 / Biz 안정화

MOBD + 인포카 Pro 구독 정책 수정 · Biz 안드로이드 버그 수정 (로그인 실패 예외·동적 뷰 제약) · 자동 연결 실차 테스트

2023.11 - 2023.12

인포카 Biz Android 합류

본앱 + Biz 앱 동시 담당 시작 · Biz 지출 관리 디자인 수정 · CS 우선순위 처리 (대표이사 지정)

2023.07 - 2023.10

앱 핵심 기능 운영

블루투스 통신 · 푸시 · 알림 센터 · 정비 항목 UX/UI 2차 개발 · 차계부+정비 내부 테스트

2022.10 - 2023.06

차계부 시스템 리뉴얼

단일 테이블 → 정규화 다중 테이블 · 무손실 마이그레이션 · 주유소 API · 구글 맵 위치 기반 · 다단위 글로벌

2022.08 - 2022.09

입사 초기 · 터미널 프로그램

블루투스 connection · 소켓 통신 · UI · 로그 저장 (이메일·메모장). OBD 통신 기초 학습기.

기술 보유.

현업 4년간 직접 다뤄온 도구·언어·아키텍처. 컬러 표기는 주력 기술.

언어와 아키텍처

01

언어 · 디자인 패턴 · 의존성 주입

Kotlin

Java

MVVM

Hilt

Coroutines / Flow

레포지토리 패턴

클린 아키텍처

UI와 디바이스

02

화면 · 디바이스 확장

Jetpack Compose

XML 레이아웃

Car App Library

멀티 디바이스 대응

데이터 시각화

머티리얼 3

데이터와 통신

03

저장소 · 통신 · 비동기

Room (SQLite)

DataStore

SharedPreferences

Firebase Remote Config

OBD2

Retrofit

SharedFlow / StateFlow

LiveData

AI 활용과 협업

04

AI 워크플로우 · 협업 도구

Claude Code

Notion MCP

Figma MCP

knot (지식 vault)

Git

슬랙

노션 · 피그마

스크림

광고 · 결제 · 미디어이션 · AdMob · AppLovin · ironSource · Mintegral · Pangle · Unity Ads · UMP (GDPR/CCPA)

PROJECT 03에서 5개 네트워크 워터폴 + 실시간 원격 제어 구현 경험

참여 프로젝트.

최신순 5개 — 다음 페이지부터 프로젝트당 3~4장씩 상세 설명.

P.01 · 현재

2026.01 - 2026.05

AI 워크플로우 기반 개발

요구사항→명세→구현→검
증을 AI 도구와 함께 · 레거
시 점진 전환

P.02

2025.09 - 2025.12

프리셋 대시보드 (OBD 차량 데이터)

B2B 기능의 B2C 이식 · 5
종 엔진 × 3종 대시보드

P.03

2025.03 - 2025.05

광고 미디어이션 전환

단일 AdMob → 5개 네트
워크 워터폴 · Remote
Config

P.04

2024.10 - 2025.02

안드로이드 오토 플랫폼 확장

Car App Library 신규 ·
앱-오토 화면 동기화

P.05

2022.10 - 2023.06

차계부 시스템 DB 리뉴얼

단일 테이블 → 정규화 · 무
손실 마이그레이션

도메인 흐름

A

차량 · 데이터 · 운전 환경

OBD2 프로토콜

차량 ECU 통신

블루투스 페어링

제조사 데이터

표준 PID

안드로이드 오토

기술 흐름

B

언어 · 아키텍처 · 비동기

Kotlin · Java 혼용

MVVM

Hilt · Coroutines / Flow

Jetpack Compose

Room · DataStore

Firebase Remote Config

일관된 원칙

C

매번 지킨 가치

레거시 점진 전환

라이프사이클 고려

설정 중앙 관리

멀티 디바이스 대응

데이터 보존 마이그레이션

사용자 1,000만+

AI 워크플로우 기반 개발.

요구사항 정의부터 검증까지, AI 도구를 실제 작업에 통합한 개발 방식.
단발성 코드 생성이 아니라 프로젝트 지식·기획·디자인을 AI가 함께 참조하도록 환경을 만들어 일관되게 진행.

기간	2026.01 ~ 진행 중
대상	인포카 안드로이드 실무
역할	AI 워크플로우 구축·정착, 레거시 점진 전환 (안드로이드)
도구	Claude Code · Notion MCP · Figma MCP · knot(지식 vault) · Kotlin / Java

핵심

이 방식의 뼈대

● 작업 흐름

4

단계

요구사항 → 명세 → 구현 → 검증

● AI 연동

Notion · Figma

MCP + knot 지식 vault 참조

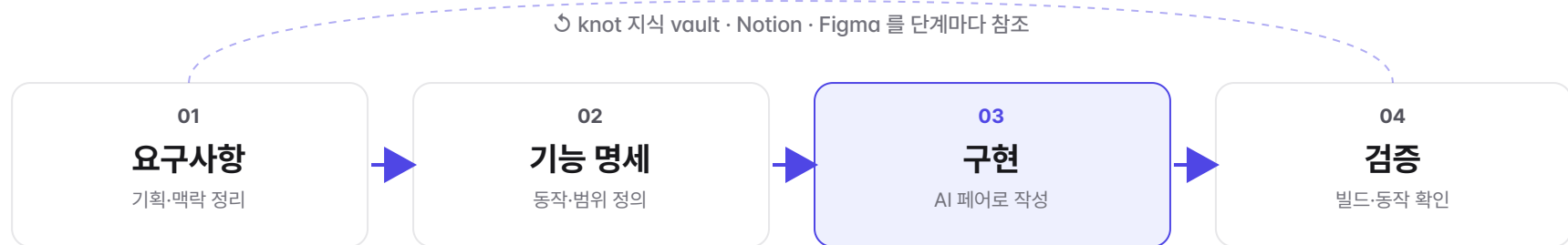
● 전환 방식

점진 전환

레거시 보존 · 신규부터 적용

- 프로젝트 지식(도메인·구조·규칙)을 **knot 지식 vault**로 정리해, AI가 일관된 맥락으로 작업하도록 구성
- **Notion MCP**로 기획·요구사항을, **Figma MCP**로 디자인을 AI가 직접 참조
- 요구사항 → 기능 명세 → 구현 → 검증을 **한 흐름으로 연결** — 단계 산출물이 다음 단계 입력이 됨
- 레거시(MVC·Java)는 그대로 두고, 신규·수정 기능부터 **Kotlin 구조**로 점진 전환

요구사항부터 검증까지.



각 단계의 산출물이 다음 단계 입력이 됨

산출물 각 단계별 결과물

01 · 요구사항
무엇을 · 왜
해결할 문제 · 사용자 시나리오 · 우선순위

02 · 기능 명세
어떻게 동작
화면 · 데이터 · 상태 흐름 정의

03 · 구현
코드 + 커밋
정의대로 작성 · 변경점 정리

04 · 검증
동작 확인
빌드 · 주요 시나리오 · 크래시 점검

A 한 흐름으로 연결

요구사항부터 검증까지 끊기지 않게 이어, 맥락이 단계마다 유지됨. 앞 단계 결과가 다음 단계 입력이 되어 같은 설명을 반복할 필요가 줄어들.

B 고치기 전에 영향 범위 확인

코드를 수정하기 전에 관련된 부분을 함께 살펴, 빠뜨리는 곳이나 사이드이펙트를 줄임. 동작하는 레거시는 그대로 보존.

맥락을 갖춘 AI 협업.

AI가 추측하지 않도록 프로젝트의 지식·기획·디자인을 직접 참조할 수 있게 연결. 매번 같은 설명을 반복하지 않고 일관된 결과를 얻는 것이 목적.

knot · 지식 vault

A

- 도메인 용어 (OBD2·ECU·블루투스)
- 아키텍처·프로젝트 구조
- 코딩 규칙·컨벤션

Notion MCP

B

- 기획서·요구사항
- 작업 맥락·배경
- 우선순위

Figma MCP

C

- 디자인 화면
- UI 스펙·치수
- 컴포넌트

Claude Code · 실행

D

- 위 맥락을 묶어 요구사항→명세→구현→검증 실행
- 반복 작업을 스킬로 표준화

적용 방식

E

- 기능마다 같은 흐름을 적용
- 새로 만들거나 고치는 기능부터 우선 도입

설계 원칙 · 맥락을 한 번에 다 주기보다, 단계마다 **필요한 것만** 골라 제공해 AI가 핵심에 집중하도록.

AI가 추측 대신 실제 자료를 근거로 작업하게 하는 것이 목표

점진 전환.

동작하는 레거시는 그대로 두고, 새로 만들거나 고치는 기능부터 새 구조를 적용. 한 번에 다 뒤집지 않고 안정적으로 공존시키는 방식.

기존 (AS-IS)

MVC (Java)

역할 뒤섞임

화면 코드가 화면·로직·데이터를 한꺼번에 들고 있어, 한 곳을 고치면 어디까지 영향이 가는지 파악이 어려움. 상태가 여러 곳에 흩어져 공유 되고, 테스트를 붙이기도 어려움.



목표 (TO-BE)

Kotlin · MVVM · Hilt

역할 분리 · 흐름 명확

화면은 상태만 그리고, 로직·데이터는 분리. 상태 흐름이 한 방향으로 명확해 변경 영향이 잘 보임. 신규 기능부터 적용하고, 레거시와는 경계에서 연결.

전환 원칙

네 가지 원칙

- 레거시 보존 — 잘 동작하는 코드는 건드리지 않고 경계로 분리
- 신규부터 적용 — 새 기능은 처음부터 Kotlin·MVVM·Hilt로 작성
- 경계에서 연결 — 레거시 ↔ 신규 사이에서 데이터 형식을 변환
- 고치기 전 영향 확인 — 변경 전 관련 범위를 함께 점검해 누락·사이드이펙트를 줄임

현황

진행 중 (2026 상반기 기준)

- 새로 만들거나 고치는 기능은 새 구조(Kotlin·MVVM)로 작성하며 점진 확장 중
- 요구사항 → 검증을 한 흐름으로 진행하는 방식을 실무에 정착시키는 중
- 한 번에 전부 바꾸지 않아, 큰 회귀 위험 없이 단계적으로 전환

프리셋 대시보드 (OBD 차량 데이터).

B2B(인포카 비즈) 자산을 B2C(인포카)에 이식·확장하면서, 하드코딩된 센서 매핑을 사용자 선택형 + 엔진 타입별 5종 프리셋으로 재설계.

● 엔진 타입

5종

가솔린·디젤·LPG·전기·수소

● 대시보드

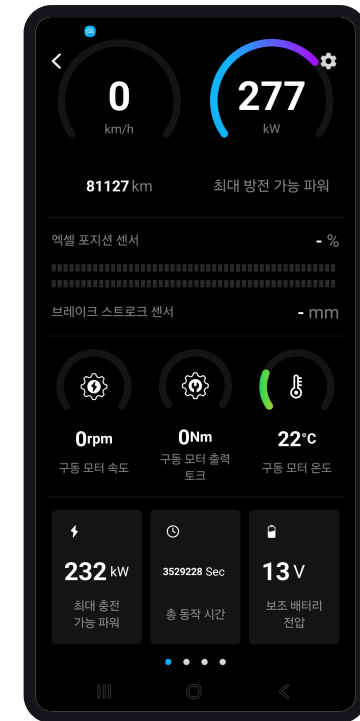
3종

주행 · TPMS · 배터리

● 레이아웃

4종

폰·태블릿 × 가로·세로

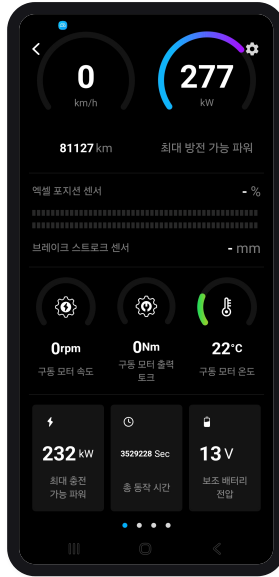


▲ 주행 대시보드 (실제 화면)

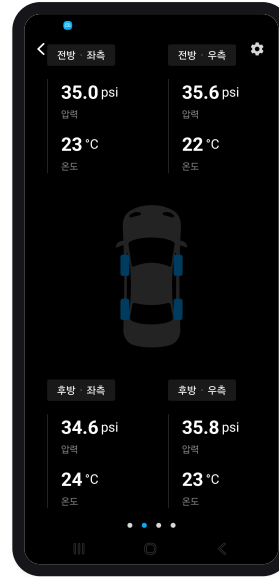
설계 방향 · 데이터 항목을 무작정 늘리지 않고, **엔진 타입에 맞는 항목만** 골라 보여주도록 구성. 디젤은 DPF·배출 관련, 전기차는 배터리 상세처럼 차량마다 필요한 데이터만 묶어서 노출.

기간	2025.09 ~ 2025.12 · 약 4개월
역할	정보 구조 설계 · 안드로이드 단독 구현 · B2B↔B2C 코드 통합
기술	Kotlin, Java, Jetpack Compose, DataStore, Room, MVVM, SharedFlow / LiveData

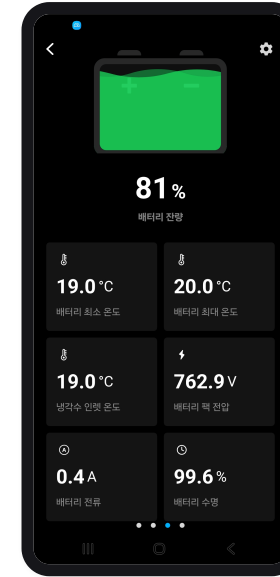
엔진 타입별 데이터 구성.



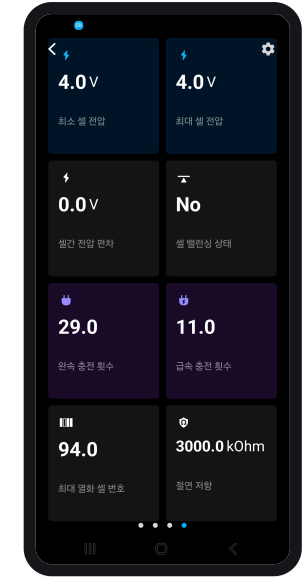
주행 대시보드
속도·RPM·연료·온도



타이어 (TPMS)
4륜 압력·온도·경고



배터리 (메인)
SOC·전압·전류·SOH



서브 배터리 (EV)
셀 편차·절연저항

A 사용자 선택형 항목

한 항목(예: 엔진 RPM)에 해당하는 센서 값이 여럿일 때 사용자가 직접 선택.

- RPM 센서가 여러 개인 차량 → 사용자가 선택
- 전기차 모터 RPM(전륜·후륜) 선택
- 타이어 위치(TPMS) 차량별 재설정

B 임계치 자동 환산

타이어 온도 80°C·압력 25psi 고정 → 사용자 설정 + 단위 자동 환산.

- 80°C ↔ 176°F (자동)
- 25 psi ↔ 1.72 bar (자동)
- 온도·압력 항목 자동 인식

ECU에서 위젯까지.



설정 항목 정의를 한 곳에서 관리

항목 · 기본 단위 · 연료타입 · 임계치 등을 한 곳에 모아 정의. 새 항목을 추가할 때 한 군데만 수정하면 됨 — 예전엔 서너 군데를 고쳐야 했음.

통합 공통 인터페이스로 B2C·B2B 통합

공통 인터페이스 하나에 B2C·B2B 두 구현을 둬. 화면 코드는 빌드 종류만 한 번 확인한 뒤 같은 방식으로 사용 — 두 서비스의 차이를 안쪽에서 흡수.

구조 레이어별 역할

레이어	역할
설정	모든 항목 정의를 한 곳에 모아 중앙 관리
데이터 저장소	B2C·B2B 공통 인터페이스 + 각 서비스 구현
화면 상태 관리	항목별 실시간 값 관리 · 동시 접근 안전 처리 · 설정 자동 복구

주요 기술 처리.

A · 설정 자동 복구

무효 항목 자동 복구

제조사 데이터가 갱신돼 기존 설정이 안 맞게 되면 자동으로 다시 연결. 사용자 설정 손실 없음.

B · 데이터 마이그레이션

설정 형식 자동 전환

예전 설정 형식을 새 형식으로 자동 변환·저장. 앱 업데이트 후에도 사용자 설정 유지.

C · 임계치 단위 환산

psi↔bar · °C↔°F

단위를 바꾸면 임계치 값도 자동으로 함께 환산. 온도·압력 항목 자동 인식.

D · 표준값 대체

제조사 데이터 미보유 대응

제조사 전용 값을 못 읽으면 표준 값으로 자동 대체. 제조사 데이터가 없는 사용자도 가능한 항목 표시.

E · 동시 접근 안전

데이터 경쟁 방지

여러 곳에서 동시에 접근해도 충돌 없이 처리 · 같은 데이터는 한 번만 조회.

F · 동적 폰트 크기

화면에 맞춰 자동 조정

영역 너비에 맞춰 글자 크기를 단계적으로 축소. 폰·태블릿 모두 잘리지 않음.

정량 효과

Before / After 비교

지표	Before	After	효과
바인딩 코드 길이	464줄	258줄	▼ 44%
신규 항목 추가 비용	서너 군데 수정	한 곳만 수정	▼ 70%+
지원 엔진 타입	2종 (가솔린·디젤)	5종 (+LPG·EV·수소)	▲ 2.5×
레이아웃 분기	1종	4종 (폰·태블릿 × 가로·세로)	▲ 4×
사용자 매핑 옵션	0 (하드코딩)	항목별 N:1 선택	신규
B2C·B2B 코드	화면 코드 2벌	공통 인터페이스 1벌 + 구현 2벌	통일

광고 미디어이션 전환.

단일 AdMob → 5개 네트워크 워터폴 미디어이션. Firebase Remote Config 9개 항목으로 무배포 실시간 정책 제어.

기간	2025.03 ~ 2025.05 · 약 3개월
인원	4명 (PM · 디자이너 · iOS · 안드로이드)
역할	광고 미디어이션·원격 제어 시스템 설계·구현 (안드로이드)
기술	Kotlin, Java, AdMob, AppLovin · ironSource · Mintegral · Pangle · Unity Ads, Firebase Remote Config, UMP, CompletableFuture

배경 단일 AdMob의 한계 (30일 기준)

● 전면 광고 수익

\$869 /30일

405,778 노출 · eCPM \$2.14

● 네이티브 eCPM

\$1.42

전면 대비 ▼34% — 광고 피로도

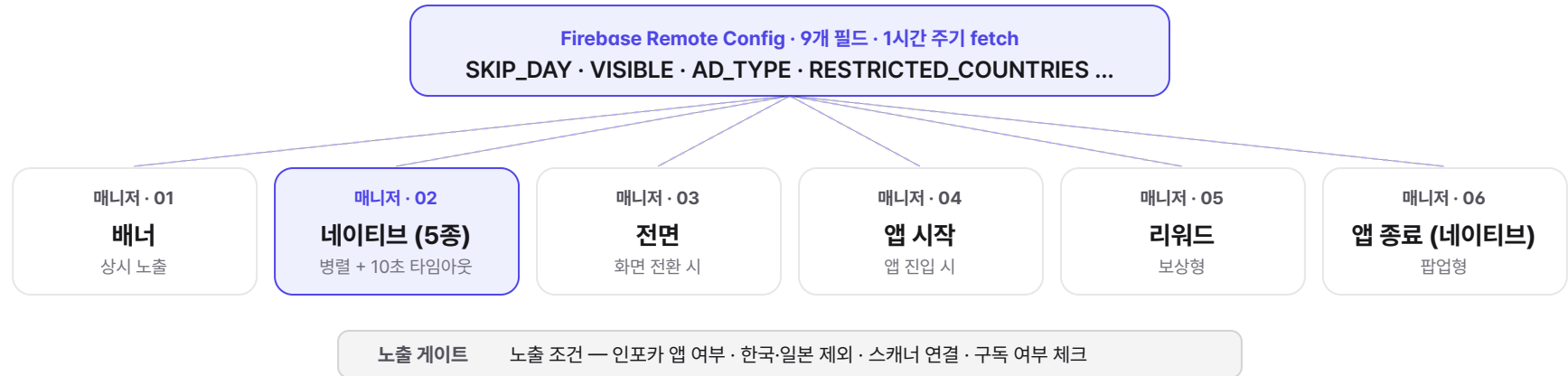
● 활성 사용자 체류

38 분

배너 광고 부재 → 지속 수익 0

- 국가별 노출 편중 — 브라질·멕시코 노출 많음, 고eCPM 미국·스위스·호주는 노출 부족
- 한국·일본 광고 부재 — 자체 광고만 운영, AdMob 수익 0
- 네이티브 팝업의 UX 저해 — CTR·eCPM 동반 하락. 자연스러운 배너 전환 필요

광고 타입별 매니저 + 무배포 정책.



A 광고 단위 ID 12개 세분화

위치별 광고 단위 분리 → CTR·eCPM 추적 가능.

- 네이티브 5종 (메인·사이드·바텀시트·자체배너·대시보드)
- 배너·전면·앱시작·리워드·앱종료

B Remote Config 9개 동적 제어

서버 배포 없이 광고 정책 변경.

- SKIP_DAY · DIALOG_VISIBLE · APP_OPENING
- REWARD_AD_SHOW · DASHBOARD_AD_TYPE
- RESTRICTED_COUNTRIES = ["KR", "JP"]

국가 별 정책 자체 광고 + AdMob 통합 분기

한국 · 일본

자체 광고 우선

자체 배너 영역에 AdMob 네이티브 광고를 자연스럽게 섞어 노출. 표준 광고는 미노출.

해외 · 일반 사용자

네이티브 + 배너

메인·차계부·진단·기록 페이지에 네이티브 배너. 하단 텍스트 형 배너 추가.

해외 · 타사 스캐너 연결

전체 광고 활성화

전면·앱시작·앱종료·팝업 + 목록 5개당 네이티브 광고 노출 (구독자 제외).

eCPM 워터폴 + 병렬 로딩 최적화.

워터폴 5개 네트워크 + AdMob 기반

▲ 고 eCPM

폴백 ▼



사업 지표 2023 → 2024 매출 변화 (회사 전체 지표)

본인 기여 · 안드로이드 SDK 통합 · 매니저 구조 · UMP 자동화 · Remote Config 정책 제어 (매출 증가는 정책·사업 결정 등 복합 요인)

● 앱내 광고 매출

▲ 475 %

6.7M원 → 39.1M원 (Android+iOS 합산)

● 구독·인앱 (Android)

▲ 11 %

179M원 → 200M원

● 일 평균 체류

▲ 18 %

32분 → 38분 (MAU 1인당)

기술 효과 Before / After 시스템 변화

지표	Before	After	효과
미디어이션 네트워크	1개 (AdMob 단일)	5개 + AdMob 기반	▲ 5×
광고 단위 ID	~7개	12개 세분화	▲ 1.7×
정책 변경 속도	스토어 재배포	Remote Config 실시간	분 단위
한국·일본 광고	없음 (자체광고만)	자체 + AdMob 통합	+α 수익

안드로이드 오토 플랫폼 확장.

Car App Library로 신규 안드로이드 오토 앱 개발. 앱과 오토 화면을 동기화하고, 앱을 종료해도 오토가 단독으로 동작하도록 구성.

● 화면

7종

메인·연결·대시보드·표준·안내·데모

● 안내 페이지

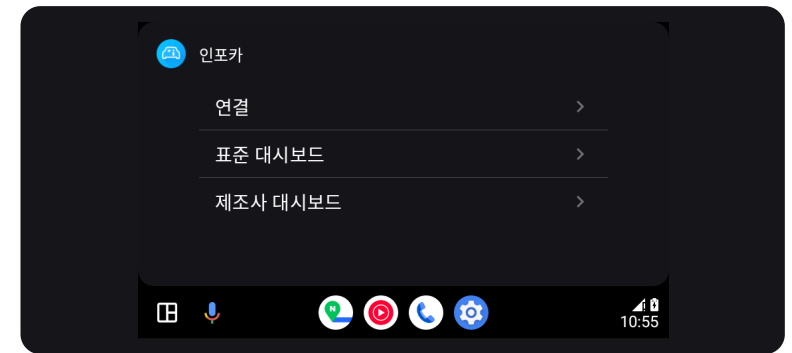
11개

권한·연결·구매·다운로드·검색

● OBD 매개변수

31개

표준 매개변수 · 데모 시뮬레이션



▲ 안드로이드 오토 메인 화면

정책 제약 인식 · 안드로이드 오토 / 카플레이는 운전 중 안전을 위해 **리스트·페인·메시지 템플릿** 3종만 허용. 그래픽 위젯·계기판은 사용 불가. → 정보 위계 압축 + 안내 페이지 11종으로 가이드 완성도 보강.

A 화면 구성 — 운전 안전 우선

- **최상위 메뉴** — 연결 · 표준 대시보드 · 제조사 대시보드 (구매 시 노출)
- **대시보드 페이지** — 앱 설정 항목과 동기화된 페이지 카운트
- **안내 페이지** — 권한·연결·구매·다운로드 등 11가지 상태별 가이드

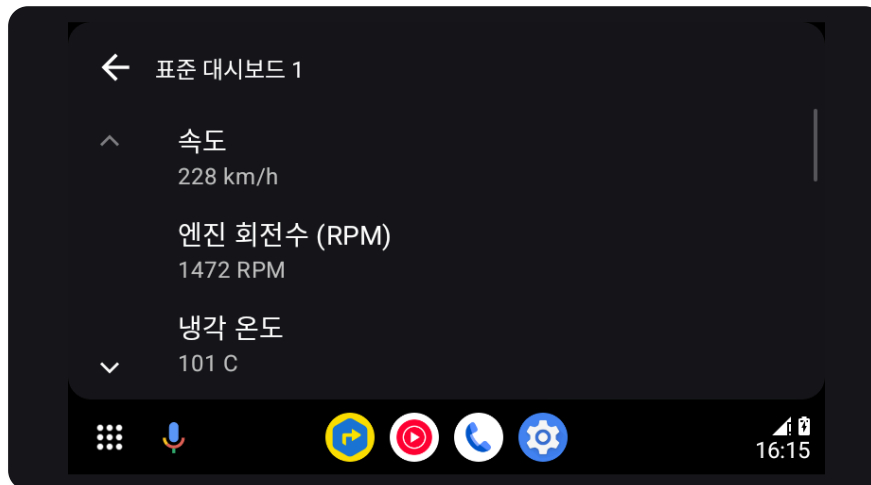
B 차량 정보 표시 (연결 화면)

- | | |
|---------------|----------|
| 01 차량 이름 | 02 제조사 |
| 03 모델 | 04 연료 타입 |
| 05 연도 | 06 배기량 |
| 07 차대번호 (VIN) | |

앱과 오토 화면 연결.

화면 계층 7종 스크린 구성

- 메인 스크린 — 리스트 템플릿 (연결 · 표준 · 제조사)
- 연결 스크린 — 페인 템플릿 (OBD/ECU 상태 + 차량정보 7필드)
- 대시보드 스크린 — 리스트 템플릿 (페이지 동적 카운트)
- 표준 데이터 스크린 — 페인 템플릿 (실시간 매개변수)
- 안내 스크린 — 메시지 템플릿 (11가지 상태)
- 데모 화면 2종 — 스캐너 없이 시연 가능

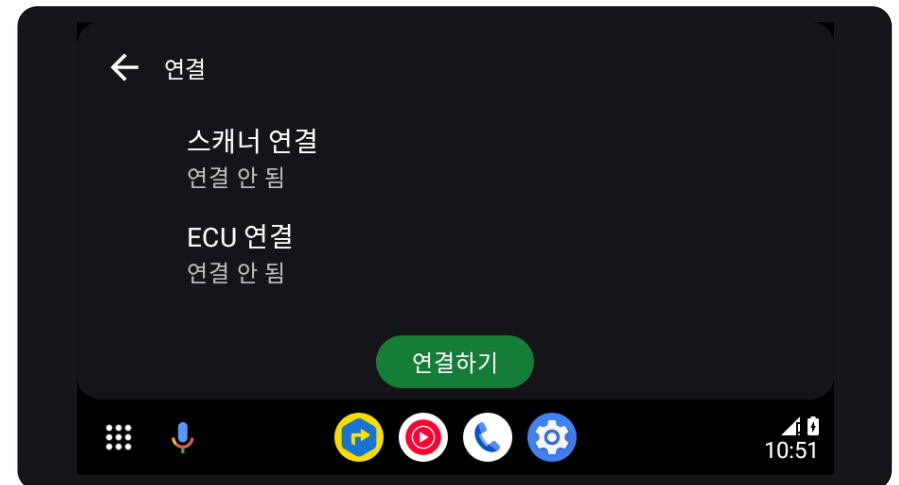


▲ 표준 대시보드 화면

동기화 앱 ↔ 오토 화면 연동

앱에서 상태가 바뀌면 오토 화면도 함께 갱신되도록 연결. UI 멈춤(ANR) 없이 처리.

- 연결 상태 변경 → 오토 연결 화면 갱신
- 대시보드 변경 → 오토 대시보드 화면 갱신
- 실시간 데이터 변경 → 표준 데이터 화면 갱신
- 상태 공유 — 앱과 오토가 같은 연결·차량 상태를 바라봄
- 독립 동작 — 앱을 종료해도 오토는 단독으로 계속 실행



▲ 연결 화면 (스캐너·차량 상태)

상태 코드 + 11개 안내 페이지.

A 연결 상태 머신 (0-3)

OBD·ECU 두 채널의 상태를 0-3 코드로 명확히. 둘 다 2일 때만 데이터 표시.

코드 0 미연결

→ 안내 스크린

코드 1 연결 중

→ 스피너 표시

코드 2 연결 완료

→ 데이터 표시

코드 3 실패

→ 오류 안내

B 11개 안내 페이지

상황별 구체적인 가이드로 첫 사용 경험 보호.

- 오토 권한 미설정 · 위치 권한 필요
- 오버레이 권한 필요
- 스캐너 먼저 연결 필요
- 제조사 데이터 구매 필요
- 제조사 프로필 다운로드 필요
- 제어유닛(ECU) 검색 필요
- 제조사 설정 진행 중 · 진단 설정
- 대시보드 항목 비어 있음 · 일반 오류

포어그라운드

앱 종료 후 단독 동작

- 위치 기반 동작 — 위치 권한이 있을 때만 위치 기반 포그라운드 서비스 사용, 없으면 대체 경로
- OS 권한 제약 대응 — 최신 안드로이드 권한 정책에 맞춰 예외 상황 안전 처리
- 앱 종료 후 유지 — 메인 앱을 종료해도 오토는 계속 데이터 갱신 (배터리 영향 최소화)
- 재진입 시 복원 — 다시 들어와도 이전 화면·상태 자동 복원, 사용자 작업 손실 없음

차계부 시스템 DB 리뉴얼.

단일 테이블 → 정규화 다중 테이블로 DB 재설계. 1,000만 사용자 데이터를 무손실 마이그레이션. 구글 맵 위치 기반 + 글로벌 단위 대응.

기간	2022.10 ~ 2023.06 · 약 9개월
인원	4명 (PM · 디자이너 · iOS · 안드로이드)
역할	DB 재설계·마이그레이션 · 위치 기반 기능 (안드로이드)
기술	Kotlin, Java, Room (SQLite), 구글 맵 API, 안드로이드 XML

배경 왜 리뉴얼했는가

- 단일 테이블의 한계 — car_log 한 테이블에 차량·주유·정비·소모품이 혼재 → 중복·확장 한계
- 무손실 요구 — 1,000만 다운로드 서비스의 누적 사용자 데이터를 절대 손실 없이 변환
- 위치 기반 확장 — 주유소 API 적용 + 구글 맵 장소 검색 통합
- 데이터 복원 안전망 — 복구 API 작업 + 마이그레이션 로직 수정으로 사용자 무손실 보장
- 글로벌 단위 대응 — 거리(km/mile), 연비(km/L, MPG), 통화 등 지역별 단위 시스템

● 기간

9개월

설계 · 마이그레이션 · 검증

● 테이블 수

4개

vehicle · fuel · maintenance · consumable

● 대상 사용자

1,000만+

누적 다운로드 · 데이터 손실 0

단일 → 4개 테이블 · 글로벌 대응.

기존 (AS-IS)

car_log (단일 테이블)

차량 + 주유 + 정비 + 소모품 혼재

type 컬럼으로 종류 구분. 중복 컬럼 다수·NULL 비율 높음. 신규 종류 추가 시 ALTER TABLE 필요.



목표 (TO-BE)

vehicle · fuel · maintenance · consumable

정규화 4테이블 · 외래키 관계

vehicle을 마스터로, 나머지는 vehicle_id 외래키로 참조. 중복 제거, 인덱스 효율 ↑, 신규 종류는 새 테이블만 추가.

전략

무손실 마이그레이션

- 구 스키마 백업 → 신 스키마 빈 테이블 생성 → 변환 INSERT
- 실패 시 트랜잭션 롤백, 백업으로 즉시 복귀
- 검증: ROW COUNT 비교 + 샘플링 데이터 무결성
- 점진 배포 — 일부 사용자 먼저 적용 후 전체 확대

확장

구글 맵 + 다단위 글로벌

- 주유소·정비소 장소 검색 + 거리순 정렬
- 거리: km · mile · yard · foot
- 연비: km/L · mile/gal (MPG), 볼륨 L · gal
- 통화·날짜 — 디바이스 로케일 기반 자동

- 구조 전환

1 → 4

테이블

정규화 다중 테이블 · 관심사 분리

- 데이터 손실

0

건

무손실 마이그레이션 검증 완료

— 마지막 페이지

읽어주셔서
감사합니다.

연락처

010-9376-2966

이메일

dkwkrhrh07@naver.com

깃허브

github.com/JayChoi07